Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To implement a Convolutional Neural Network (CNN) using TensorFlow/Keras for image classification on the CIFAR-10 dataset and evaluate its performance.

2. Learning Outcomes

1. Build and train a CNN for image classification.
2. Understand feature extraction using convolution and pooling layers.
3. Evaluate the model using standard classification metrics.
4. Analyze the effect of different CNN hyperparameters on performance.

6. Experimental Procedure

1. Load the CIFAR-10 dataset from the TensorFlow/Keras library and display the dataset dimensions.
2. Visualize ten sample images and plot the class distribution to understand the dataset.
3. Preprocess the dataset by normalizing pixel values to the range $[0, 1]$ and converting class labels into one-hot encoded vectors.
4. Construct a Convolutional Neural Network (CNN) consisting of convolutional layers, ReLU activation functions, max-pooling layers, a flatten layer, and fully connected dense layers followed by a Softmax output layer.
5. Display the model architecture using `model.summary()` and analyze the trainable parameters.
6. Compile the CNN using the Adam optimizer, categorical cross-entropy loss function, and accuracy as the evaluation metric.
7. Train the model on the training dataset using the specified batch size and number of epochs while monitoring validation performance.
8. Evaluate the trained model on the testing dataset and compute accuracy, precision, recall, F1-score, confusion matrix, and classification report.
9. Plot the training and validation accuracy curves and the training and validation loss curves to analyze the learning behaviour.
10. Visualize feature maps generated by the convolutional layers and display the learned convolution filters.
11. Construct an alternative CNN using Average Pooling and compare its performance with the Max Pooling model.
12. Compare the performance of different optimizers and analyze their effect on classification accuracy.
13. Summarize the experimental results and interpret the effectiveness of CNNs for image classification on the CIFAR-10 dataset.

Tasks

1. Load and explore the CIFAR-10 dataset.
2. Preprocess the dataset for training.
3. Build the CNN architecture.
4. Train the CNN model.
5. Evaluate the model performance.
6. Plot training and validation metrics.
7. Visualize feature maps and convolution filters.
8. Compare Max Pooling and Average Pooling.
9. Compare different optimizers.
10. Analyze the experimental results.

8. Source Code

GitHub Repository:
`https://github.com/Teena-Pranava/Deep-Learning-Lab`

9. Mandatory Plots

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix
8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

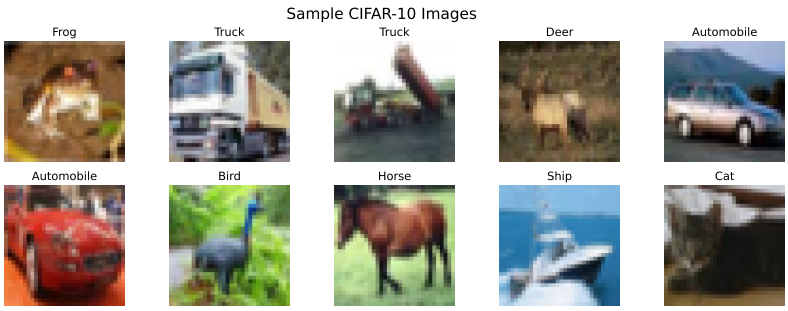

Figure 1: Sample Images

The sample images show the ten object categories present in the CIFAR-10 dataset. Each image is a 32*32 RGB image, providing sufficient visual information for the CNN to learn meaningful features for image classification.

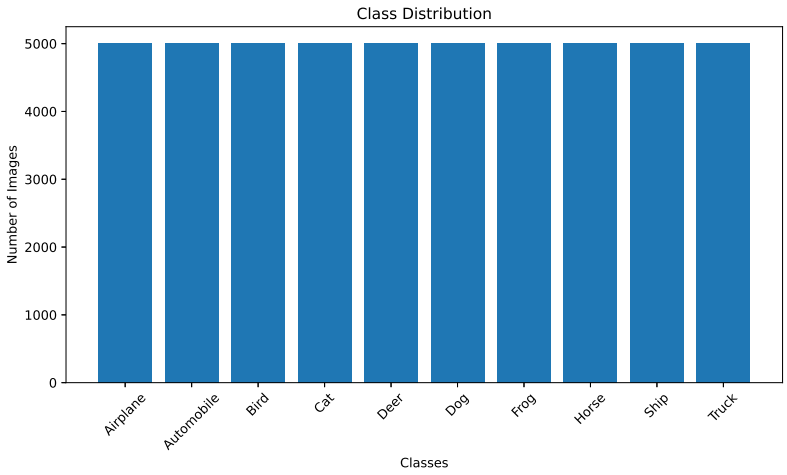

Figure 2: Class Distribution

The class distribution shows that each category contains an equal number of training images. Since the dataset is balanced, the CNN receives similar amounts of data for every class, reducing bias during training.

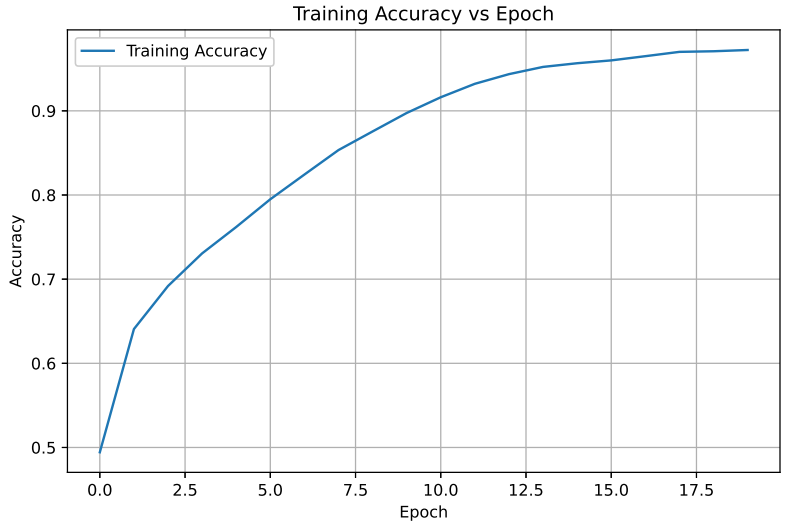

Figure 3: Training Accuracy vs Epoch

The training accuracy increases with each epoch and reaches a stable value.The improvement becomes smaller after many epochs,this indicates that the model has reached convergence.

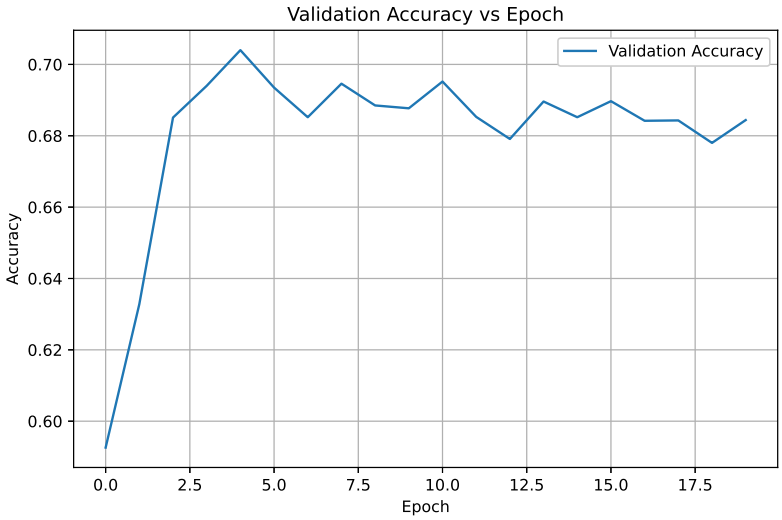

Figure 4: Validation Accuracy vs Epoch

The validation accuracy also increases with training and remains close to training accuracy. This indicates that model performs well on unseen data and there is less overfitting.

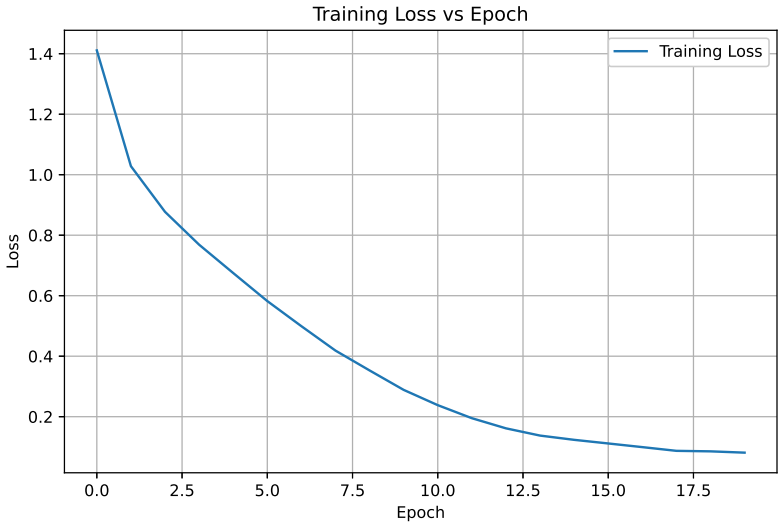

Figure 5: Training Loss vs Epoch

The training loss decreases continuously during training. The loss stabilizes after a few epochs, showing that the model has learned the important features from the dataset.

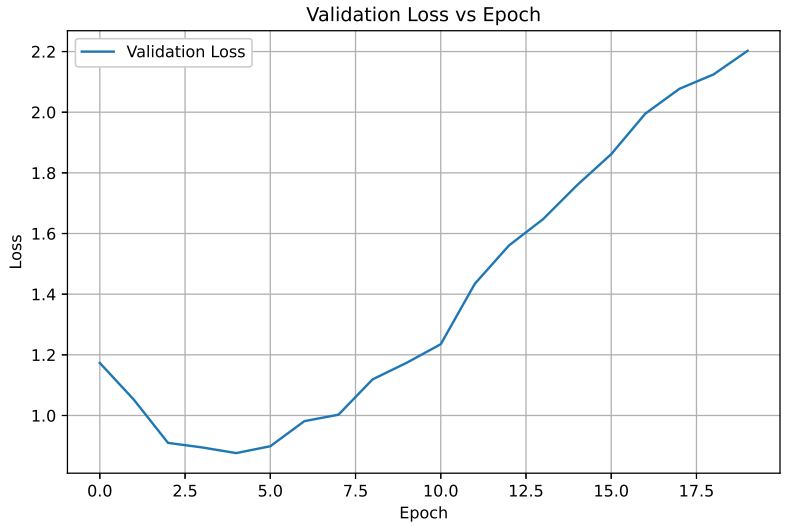

Figure 6: Validation Loss vs Epoch

The validation loss decreases during training and remains close to the training loss. This indicates that the model is able to perform well on validation data.

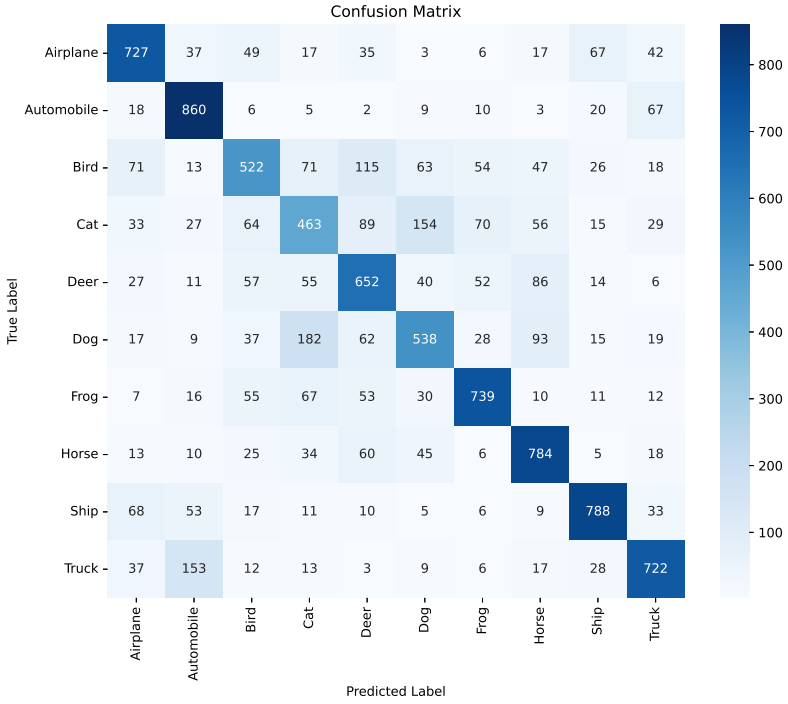

Figure 7: Confusion Matrix

The confusion matrix shows that most test images are classified correctly, as indicated by the high values along the diagonal. A few misclassifications occur between visually similar classes such as cats and dogs or automobiles and trucks. Overall, the CNN achieves good classification performance on the CIFAR-10 dataset.

Figure 8: Test Images

The test images illustrate sample inputs from the unseen testing dataset along with their predicted labels. Most of the images are classified correctly, demonstrating that the trained CNN generalizes well to previously unseen data.

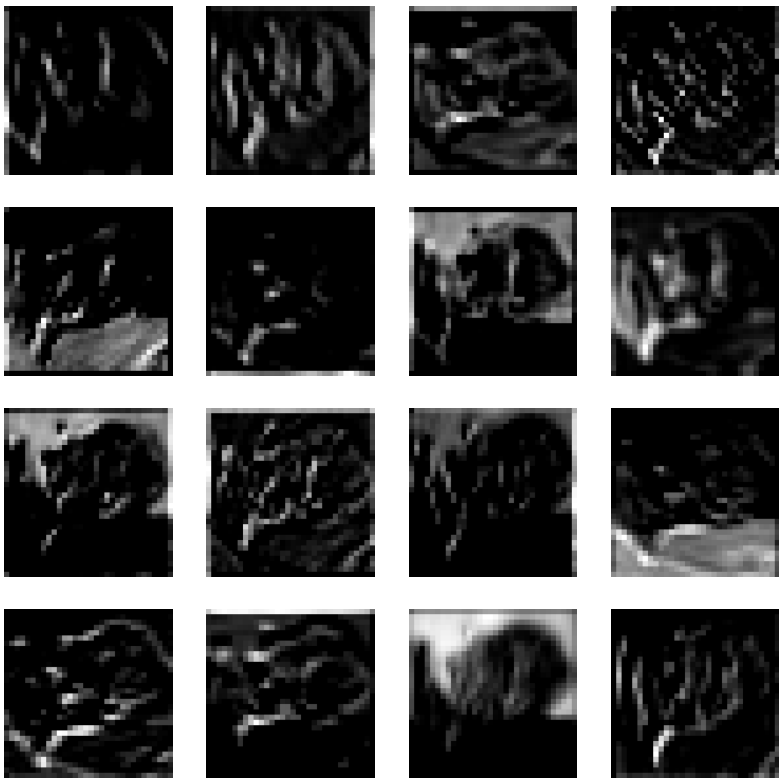

Figure 9: Feature Maps

The feature maps generated by the first convolutional layer capture low-level visual features such as edges, color gradients, and simple textures. These basic features serve as the foundation for learning more complex representations in deeper layers.

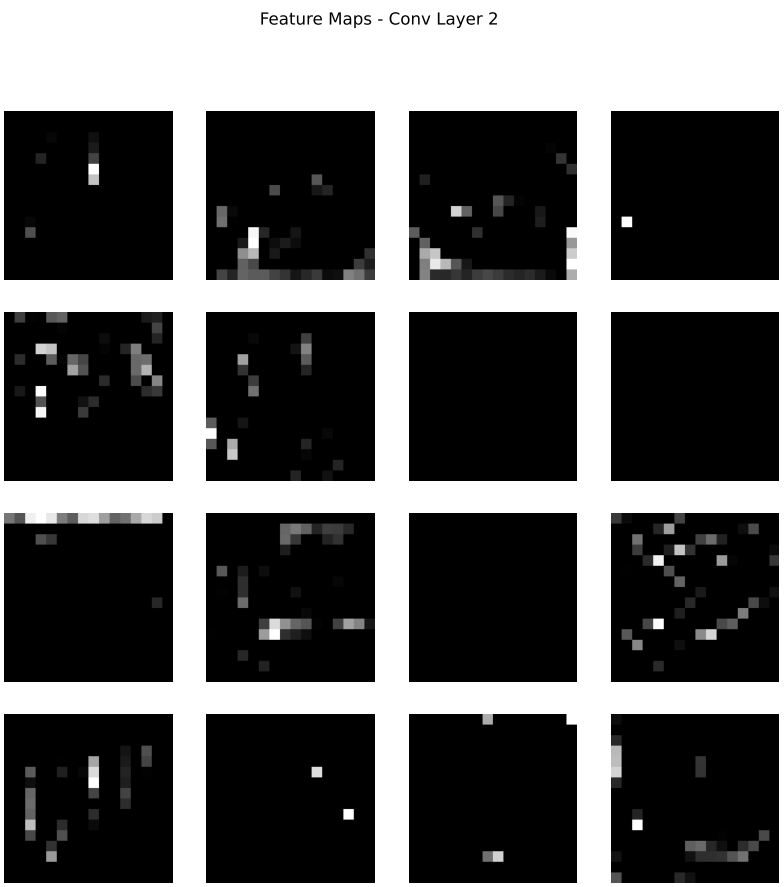

Figure 10: Feature Maps 2

The feature maps from the second convolutional layer contain more abstract and discriminative features by combining the low-level patterns extracted in the first layer. These representations help the CNN distinguish between different object categories more effectively.

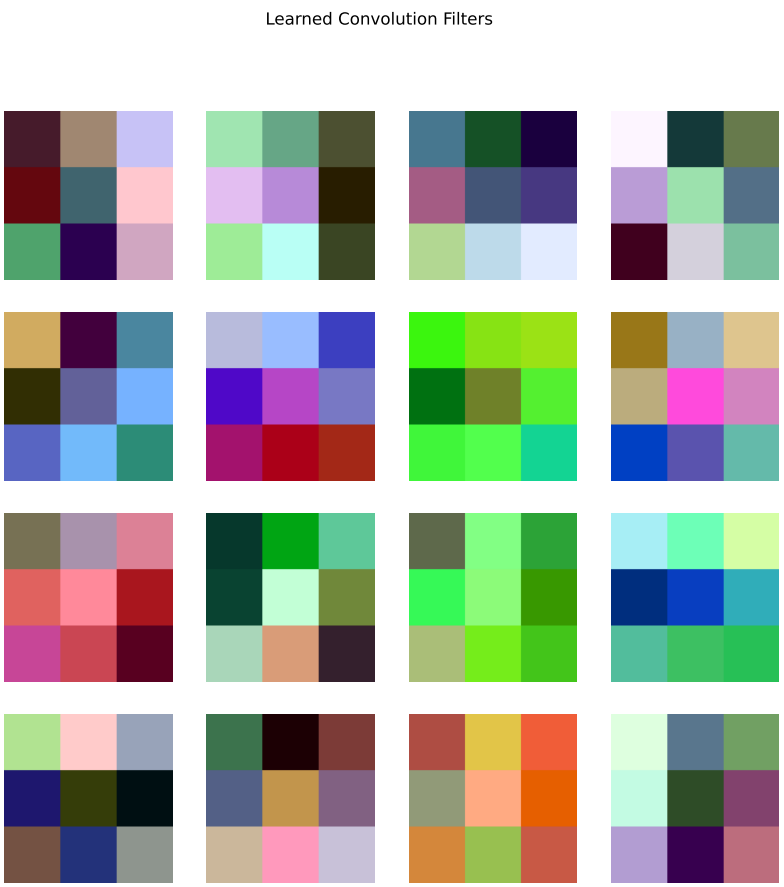

Figure 11: Filters

The learned convolution filters represent the patterns automatically discovered during training. Different filters specialize in detecting edges, textures, color transitions, and other visual characteristics that are useful for image classification.

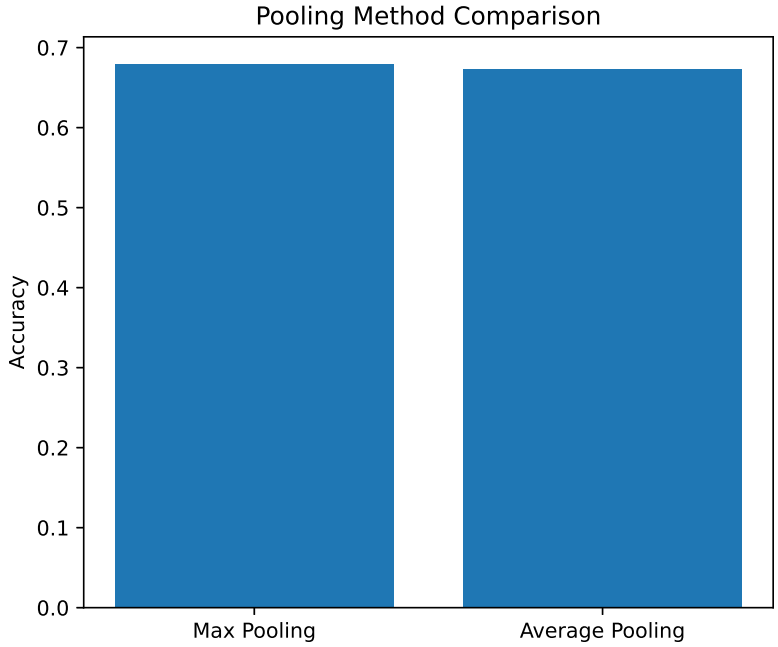

Figure 12: Pooling

The pooling comparison illustrates the effect of different pooling strategies on classification performance. Max Pooling generally provides better accuracy by preserving the most prominent features, while Average Pooling produces smoother feature representations but may lose important discriminative information.

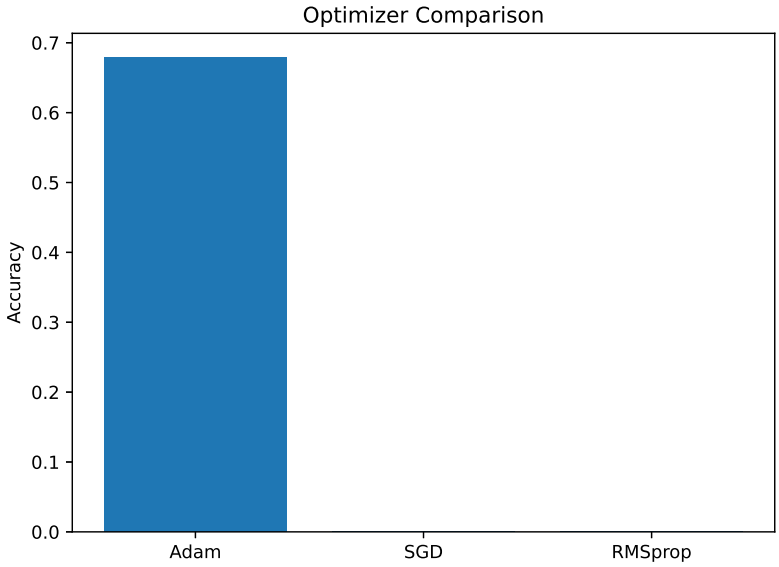

Figure 13: Optimizer Comparison

The optimizer comparison shows the effect of different optimization algorithms on model accuracy. Adam achieves the highest classification accuracy and faster convergence compared to SGD and RMSprop, making it the most suitable optimizer for this CNN model.

Dataset Summary

Property	**Value**
Dataset	CIFAR-10
Training Images	50,000
Testing Images	10,000
Image Size	$32 \times 32 \times 3$
Number of Classes	10
Color Format	RGB

Data Preprocessing Summary

Property	**Value**
Training Image Shape	(50000, 32, 32, 3)
Testing Image Shape	(10000, 32, 32, 3)
Training Label Shape	(50000, 10)
Testing Label Shape	(10000, 10)
Pixel Value Range	0.0 – 1.0
Normalization	Pixel values scaled to $[0, 1]$
Label Encoding	One-Hot Encoding

CNN Hyperparameter Summary

Hyperparameter	**Value**
Input Image Size	$32 \times 32 \times 3$
Number of Classes	10
Conv Layer 1 Filters	32
Conv Layer 2 Filters	64
Kernel Size	3×3
Pooling Layer	MaxPooling2D
Activation Function	ReLU
Optimizer	Adam
Loss Function	Categorical Crossentropy
Batch Size	64
Epochs	20
Learning Rate	0.001

CNN Architecture

Layer	Output Shape	Parameters
Conv2D	(None, 32, 32, 32)	896
MaxPooling2D	(None, 16, 16, 32)	0
Conv2D	(None, 16, 16, 64)	18,496
MaxPooling2D	(None, 8, 8, 64)	0
Flatten	(None, 4096)	0
Dense	(None, 128)	524,416
Dense	(None, 10)	1,290
Total Parameters	–	**545,098**
Trainable Parameters	–	**545,098**
Non-trainable Parameters	–	**0**

Model Performance Metrics

Metric	Value
Test Loss	2.2365
Accuracy	0.6795
Precision	0.6763
Recall	0.6795
F1-score	0.6763
Training Time (s)	1484.89

Pooling Comparison

Pooling Method	Test Accuracy
Max Pooling	0.6795
Average Pooling	0.6730

12. Report Contents

Include Objective, Learning Outcomes, Dataset, Image Preprocessing, Experimental Procedure, CNN Model Architecture, Hyperparameter Summary, Source Code, Results and Discussion, Performance Evaluation, Plots with Inference, Conclusion and References.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.